

ENSF 380

TOPIC 04: JAVA CLASSES

The main() Method

Execution starts with `main()`

Each class can have its own `main()`

...

```
public class Creature {  
    /**  
    @param args Handles an optional command-line argument (not used)  
    */  
    public static void main(String[] args) {  
        System.out.println("This is a placeholder for Creature.");  
    }  
}
```

The main() Method

01_Main

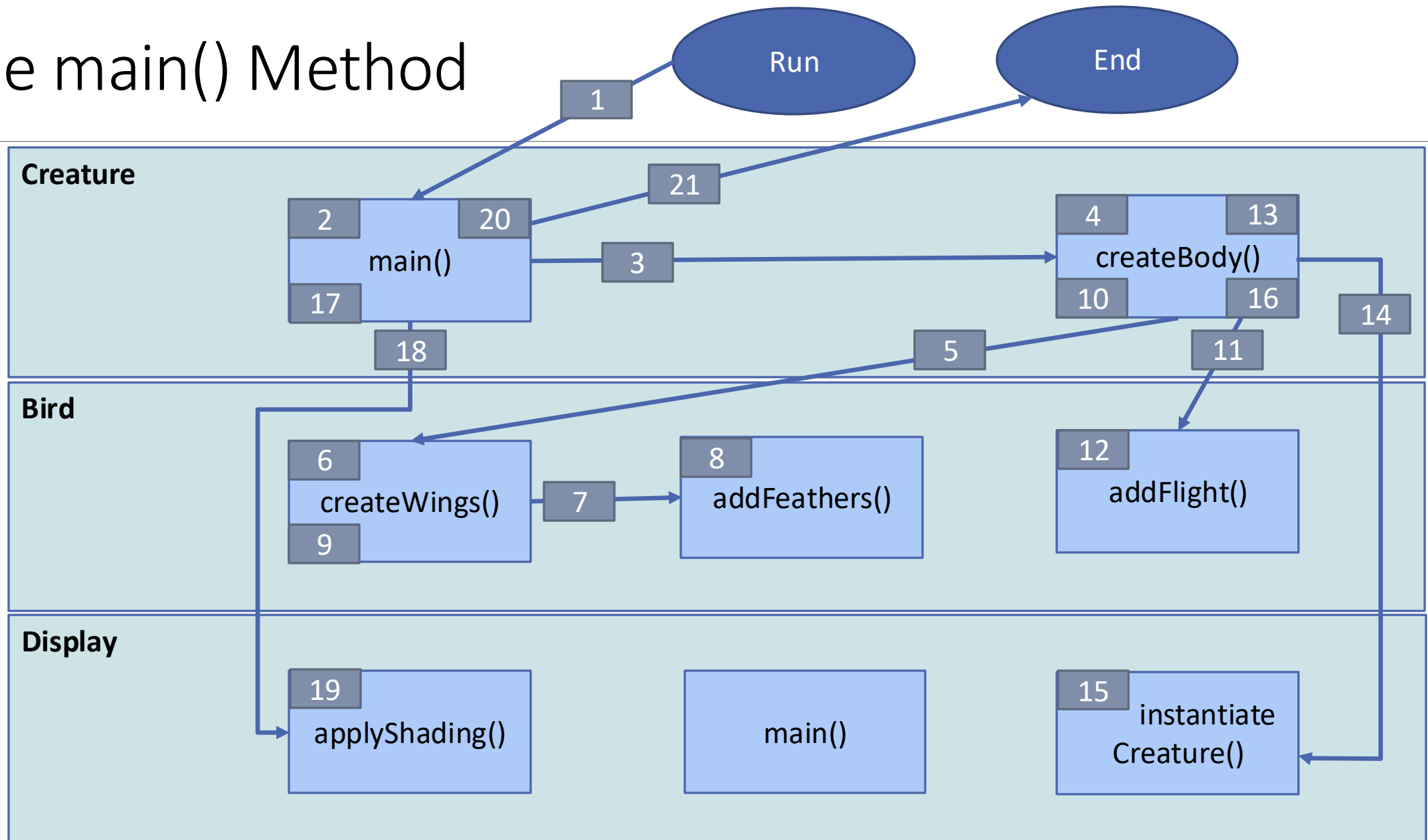
Execution starts with `main()`

Each class can have its own `main()`

...

```
public class Creature {  
    /**  
    @param args Handles an optional command-line argument (not used)  
    */  
    public static void main(String[] args) {  
        System.out.println("This is a placeholder for Creature.");  
    }  
}
```

The main() Method



The main() Method

Execution starts with `main()`

Each class can have its own `main()`

The only `main()` which is executed belongs to the program which is run

`main()` is not usually used in testing, but may be helpful at present for debugging

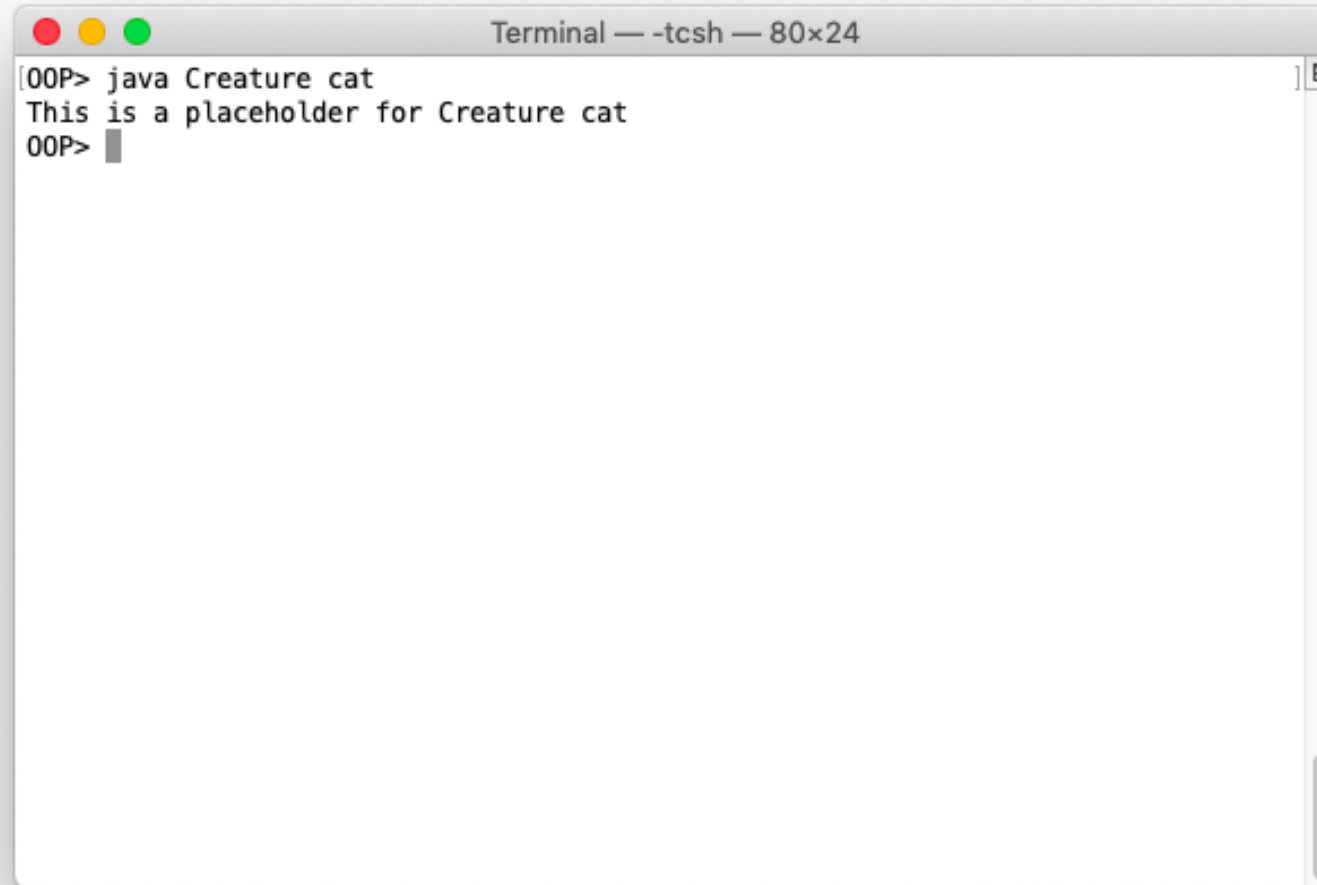
main() and a Command-Line Argument

02_CommandLineArg

...

```
public class Creature {  
    /**  
    @param args Handles an optional command-line argument (not used)  
    */  
    public static void main(String[] args) {  
        System.out.println("This is a placeholder for Creature " + args[0]);  
    }  
}
```

main() and a Command-Line Argument



```
Terminal — -tcsh — 80x24
[00P> java Creature cat
This is a placeholder for Creature cat
00P> █
```

A terminal window titled "Terminal — -tcsh — 80x24" with standard macOS window controls (red, yellow, green buttons). The terminal shows a prompt "[00P>" followed by the command "java Creature cat". The output is "This is a placeholder for Creature cat". A second prompt "00P>" is shown with a cursor (a small grey block) indicating the next input position.

Exercise 4.1

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

main() and Command-Line Arguments

03_CommandLineArg...

```
...
public class Creature {
    /**
     * @param args Handles an optional command-line argument
     */
    public static void main(String[] args) {
        System.out.print("This is a placeholder for ");
        for (int i = 0; i < args.length; i++)
            System.out.print(args[i] + " ");
        System.out.println();
    }
}
```

Introduction to Exceptions

```
...  
  
public class Creature {  
    /**  
    @param args Handles a single command-line argument  
    */  
    public static void main(String[] args) {  
        if (args.length == 0) {  
            throw new IllegalArgumentException("You must supply a command-line argument");  
        } else {  
            System.out.println("This is a placeholder for Creature " + args[0]);  
        }  
    }  
}
```

Fields and Methods

Data members (fields) and member functions (methods) are defined inside the class declaration

Instance variables and methods are associated with each instance of the class

```
...
public class Creature {
    public static void main(String[] args) {
        String myAnimalType = animal();
        System.out.println("This is a placeholder for Creature " + myAnimalType);
    }

    static String animal() {
        String animalType = "dog";
        return animalType;
    }
}
```

Fields and Methods

Note: String is also a class. Classes in Java start with a capital letter.

05_FieldsAndMethods

```
...
public class Creature {
    public static void main(String[] args) {
        String myAnimalType = animal();
        System.out.println("This is a placeholder for Creature " + myAnimalType);
    }

    static String animal() {
        String animalType = "dog";
        return animalType;
    }
}
```

Non-Primitive Data Types

Example non-primitive data types

- String
- Array
- List
- Set
- Stack

Fields and Methods

Important: Use Java standards of capitalization in all submitted work.

05_FieldsAndMethods

```
...  
public class Creature {  
    public static void main(String[] args) {  
        String myAnimalType = animal();  
        System.out.println("This is a placeholder for Creature " + myAnimalType);  
    }  
  
    static String animal() {  
        String animalType = "dog";  
        return animalType;  
    }  
}
```

```
public class Creature {
    public static void main(String[] args) {
        Animal myAnimal = new Animal();
        String myType = myAnimal.animalType();
        System.out.println("This is a placeholder for Creature " + myType);
    }
}

class Animal {
    private String animalType = "dog";

    public String animalType() {
        return animalType;
    }
}
```

Access Modifiers

Modifier	Visibility			
	Class	Package	Subclass	World
public	✓	✓	✓	✓
protected	✓	✓	✓	
default	✓	✓		
private	✓			

Exercise 4.2

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Instantiation

```
public class Creature {  
    public static void main(String[] args) {  
        Animal myPet = new Animal();  
        myPet.setAnimalType("mouse");  
        Animal myWildlife = new Animal();  
        myWildlife.setAnimalType("deer");  
  
        System.out.println("I have a pet " + myPet.getAnimalType());  
        System.out.println("I see a " + myWildlife.getAnimalType() + " outside");  
    }  
  
    class Animal {  
        private String animalType;  
  
        public void setAnimalType(String arg) {  
            animalType = arg;  
        }  
  
        public String getAnimalType() {  
            return animalType;  
        }  
    }  
}
```

Instantiation

Important: Follow the Java naming convention for getter and setter methods.

07_Instantiation

```
public class Creature {
    public static void main(String[] args) {
        Animal myPet = new Animal();
        myPet.setAnimalType("mouse");
        Animal myWildlife = new Animal();
        myWildlife.setAnimalType("deer");

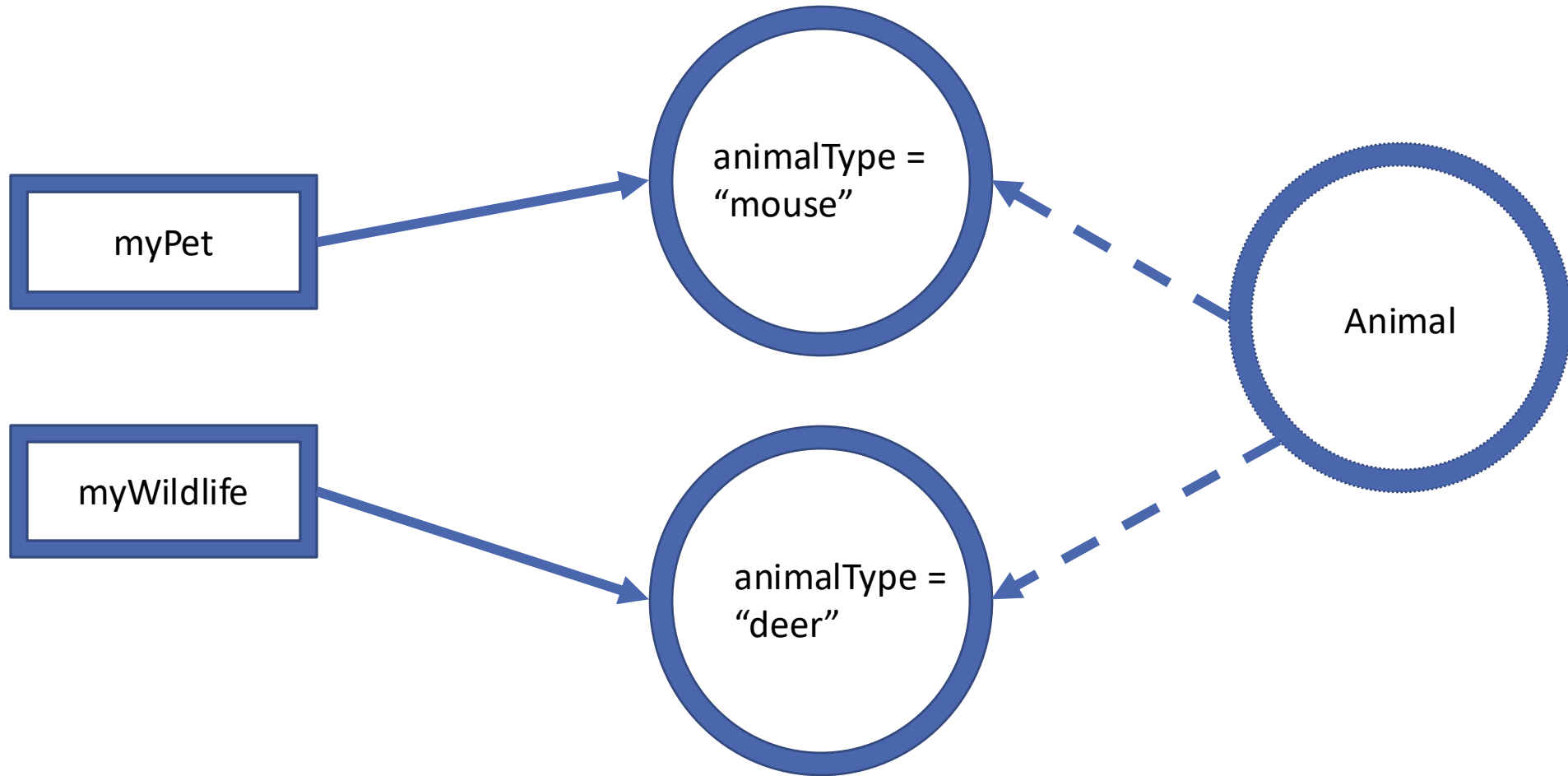
        System.out.println("I have a pet " + myPet.getAnimalType());
        System.out.println("I see a " + myWildlife.getAnimalType() + " outside");
    }
}

class Animal {
    private String animalType;

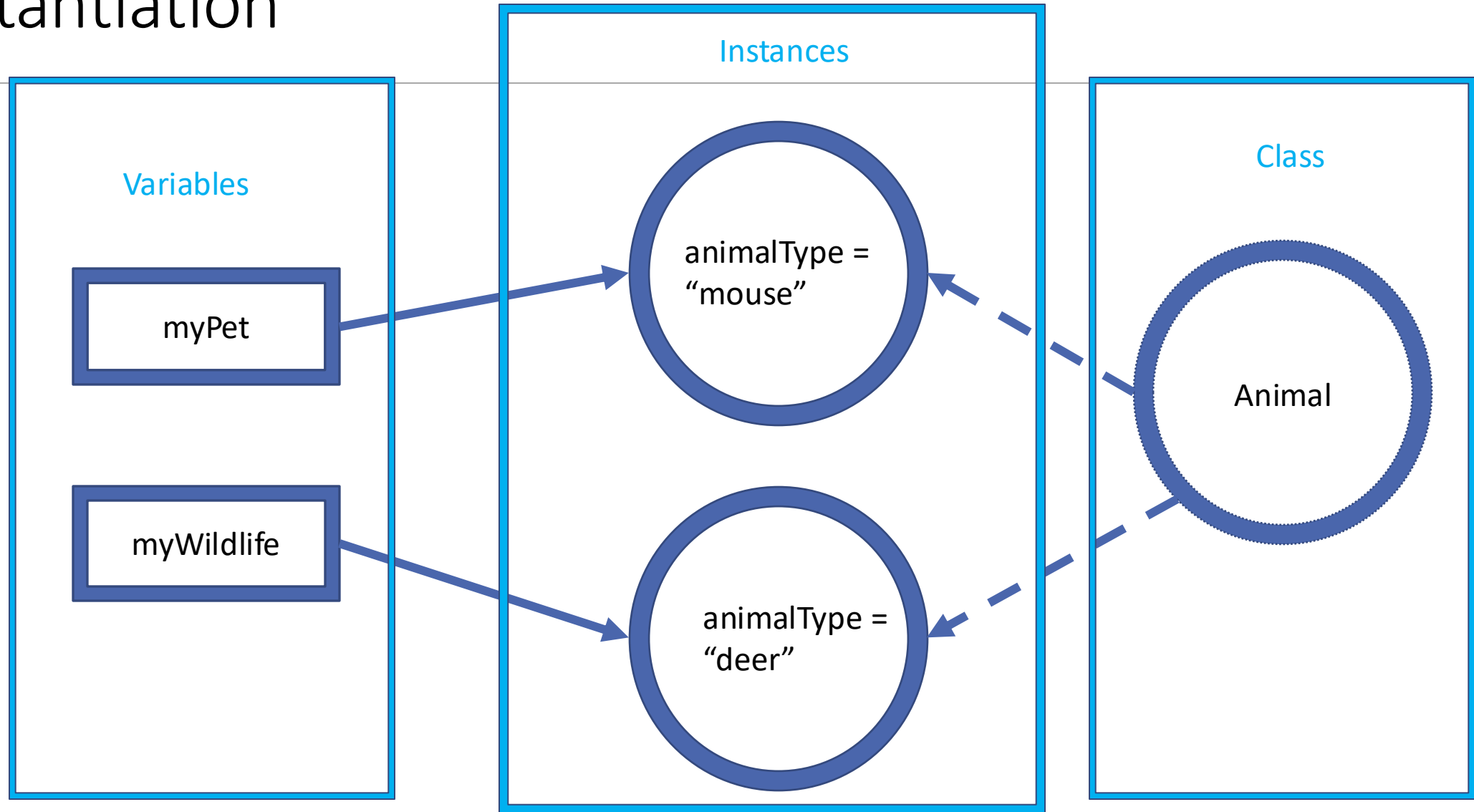
    public void setAnimalType(String arg) {
        animalType = arg;
    }

    public String getAnimalType() {
        return animalType;
    }
}
```

Instantiation



Instantiation



Avoid Inclusion

```
public class Creature {  
    public static void main(String[] args) {  
        Animal myPet = new Animal();  
        myPet.setAnimalType("mouse");  
        Animal mywildlife = new Animal();  
        mywildlife.setAnimalType("deer");  
  
        System.out.println("I have a pet " + myPet.getAnimalType());  
        System.out.println("I see a " + mywildlife.getAnimalType() + " outside");  
    }  
    class Animal {  
        private String animalType;  
  
        public void setAnimalType(String arg) {  
            animalType = arg;  
        }  
  
        public String getAnimalType() {  
            return animalType;  
        }  
    }  
}
```

Avoid Inclusion

Important: Using unintentional inclusion is a common mistake.

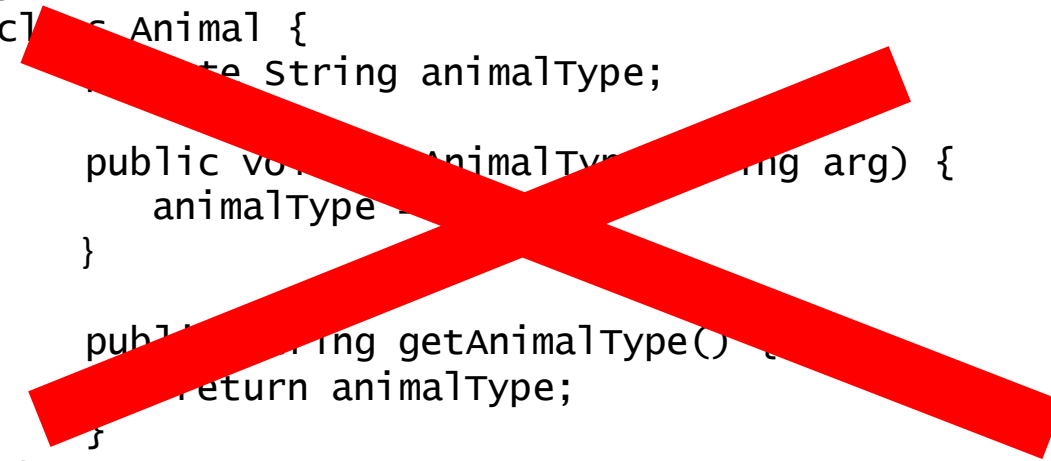
```
public class Creature {
    public static void main(String[] args) {
        Animal myPet = new Animal();
        myPet.setAnimalType("mouse");
        Animal myWildlife = new Animal();
        myWildlife.setAnimalType("deer");

        System.out.println("I have a pet " + myPet.getAnimalType());
        System.out.println("I see a " + myWildlife.getAnimalType() + " outside");
    }
}

class Animal {
    private String animalType;

    public void setAnimalType(String arg) {
        animalType = arg;
    }

    public String getAnimalType() {
        return animalType;
    }
}
```



Class Variables and Methods

08_ClassMethods

```
...
    System.out.print("I have a pet " + myPet.getAnimalType());
    System.out.println(" belonging to kingdom " + Animal.getKingdom());
    System.out.print("I see a member of kingdom " + Animal.getKingdom());
    System.out.println(", namely a " + mywildlife.getAnimalType() + ", outside");
}

class Animal {
    private String animalType;
    private static String kingdom = "animalia";

    public void setAnimalType(String arg) {
        animalType = arg;
    }

    public String getAnimalType() {
        return animalType;
    }

    public static String getKingdom() {
        return kingdom;
    }
}
```


Introduction to UML

Unified Modeling Language

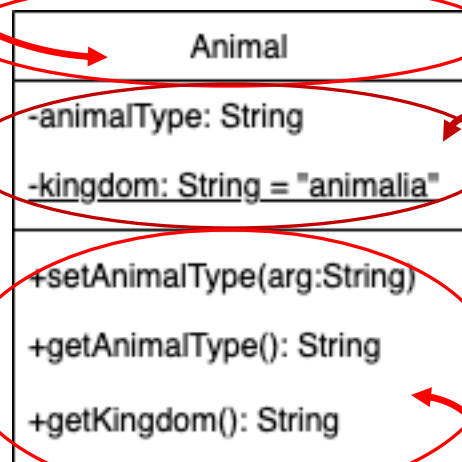
Provides standard way to visualize elements of a system design

Allows the visualization of structures, behaviors, and interactions

Class Diagram of Animal Class

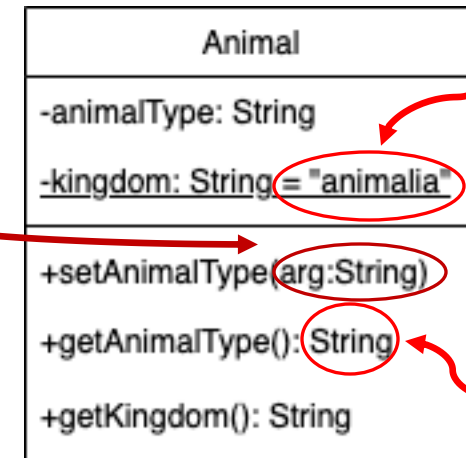
Class name section

Class fields section



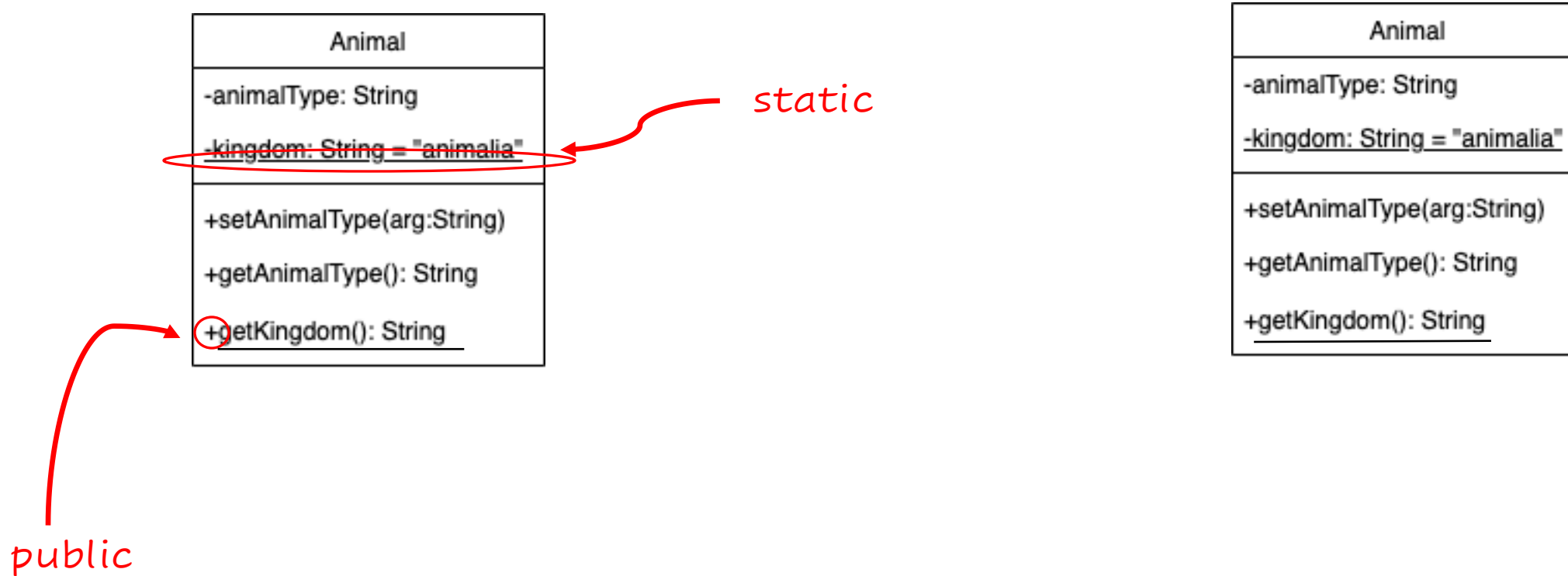
Calling argument
(name & type)

Default value



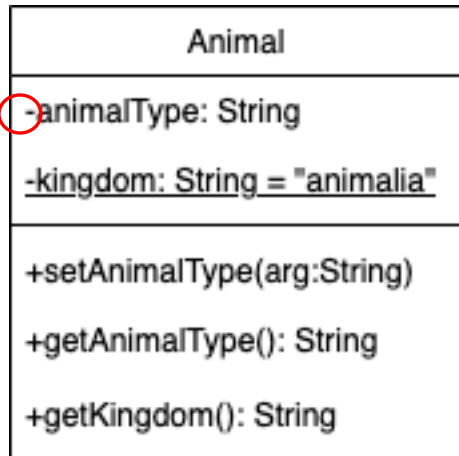
Return
value
type

Class Diagram of Animal Class



Visibility of Class Members in Class Diagram

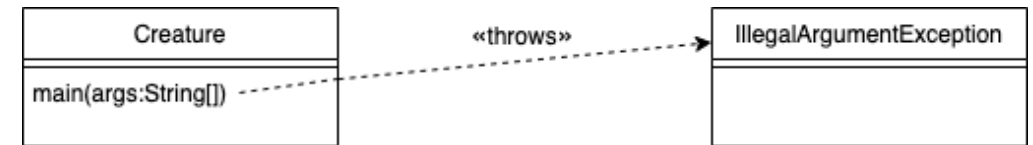
private



public	+
private	-
protected	#
package	~

Representing Exceptions in UML

```
...  
  
public class Creature {  
    /**  
    @param args Handles a single command-line arg  
    */  
    public static void main(String[] args) {  
        if (args.length == 0) {  
            throw new IllegalArgumentException("...");  
        } else {  
            System.out.println("This is a place...");  
        }  
    }  
}
```



- Draw the exception class without fields or methods (because it is an external library)
- Draw a dashed line from the method that throws the exception to the exception class name
- Label the line with «throws»

Constructors

Constructors are used to initialize the data members of a newly created object

Constructors

- Have no return type
- Must have exactly the same name as the class

If you do not supply a constructor, a default constructor is automatically provided, as in the example snippet:

```
public class Creature
{
    public Creature() {
    }
}
```

```
...  
  
public static void main(String[] args) {  
    Animal myPet = new Animal("mouse");  
    Animal myWildlife = new Animal("deer");  
  
    System.out.println("I have a pet " + myPet.getAnimalType());  
    System.out.println("I see a " + myWildlife.getAnimalType() + " outside");  
}  
  
class Animal {  
    private String animalType;  
  
    public Animal(String givenType)  
    {  
        animalType = givenType;  
    }  
}
```

Multiple Arguments

...

```
public static void main(String[] args) {
    Animal myPet = new Animal("mouse");
    myPet.setAnimalTypeKingdom("deer", "animalia");

    System.out.println(myPet.getAnimalType() + " is an " + myPet.getAnimalKingdom());
}

class Animal {
    private String animalType;
    private String animalKingdom;

    public void setAnimalTypeKingdom(String type, String kingdom)
    {
        animalType = type;
        animalKingdom = kingdom;
    }
}
```


Overloading and “This” (1/2)

11_This

```
...
public class Creature {
    public static void main(String[] args) {
        Animal myPet = new Animal();
        myPet.setAnimalKingdom("animalia");
        System.out.println(myPet.getAnimalType() + " is an " +
myPet.getAnimalKingdom());

        myPet.setAnimalTypeKingdom("deer", "animalia");
        System.out.println(myPet.getAnimalType() + " is an " +
myPet.getAnimalKingdom());
    }
}
```

Overloading and “This” (2/2)

11_This

```
...
class Animal {
    private String animalType;
    private String animalKingdom;

    public void setAnimalTypeKingdom(String type, String kingdom)
    {
        this.setAnimalType(type);
        this.setAnimalKingdom(kingdom);
    }

    public Animal(String animalType)
    {
        this.animalType = animalType;
    }

    public Animal()
    {
        this("mouse");
    }
}
```

Exercise 4.3

TEST YOUR UNDERSTANDING BY COMPLETING THE EXERCISE

Summary: Knowledge

Basics of classes

Distinguish between primitive and non-primitive data types

Standard capitalization of classes, methods, and variables

Available access modifiers and their visibility

Standard naming conventions for getter and setter methods

Distinguish between class variables and methods, and instance variables and methods

Understand the use of `main()`

Basics of UML class diagrams

Constructors

Overloading

Summary: Skills

Write a simple class, including getter and setter methods

Handle command-line arguments

Throw an exception

Instantiate an object

Depict a single class in a UML class diagram

Show throwing an exception in a UML class diagram

Pass multiple variables to a method

Use `this`

Additional Information

Related reading

- Volume 1, Chapter 4: Objects and Classes

Formatting

- Class names begin with an upper-case letter, while method and variable names begin with a lower-case letter. Subsequent words which form the name are written using CamelCase, e.g., `LibraryBook` (a class), `getAuthorName` (a method). Constants should use all capital letters, and underscores between words, e.g., `MINIMUM_TEST_SCORE`.

Access modifiers

	Different class, same package	Different package, subclass	Unrelated class, same module	Different module and p1 not exported
<code>package p1;</code> <code>class A {</code>	<code>package p1;</code> <code>class B {</code>	<code>package p2;</code> <code>class C extends A {</code>	<code>package p2;</code> <code>class D {</code>	<code>package x;</code> <code>class E {</code>
<code>private int one;</code> <code>int two;</code> <code>protected int three;</code> <code>public int four;</code>				
<code>}</code>	<code>}</code>	<code>}</code>	<code>}</code>	<code>}</code>